

Organizing a C/C++ Project

Course Document

How a C/C++ project folder is organized, in Eclipse CDT and on disk

In this course, unless an assignment says otherwise, you may use any IDE or editor to write your C or C++ code, such as Eclipse, VS Code, Neovim, Emacs, or even a plain text editor. Regardless of the editor, every project should be organized the same basic way.

Unlike Java, C and C++ do not have a package system. The compiler does not care where a source file lives on disk, and there is no statement inside a .cpp or .h file that has to match a folder path. The layout below is a convention for keeping your project easy to navigate and easy to grade, not something the language requires.

Recommendation: Until you are comfortable with project organization and source control with GitHub, I recommend using Eclipse CDT, the C/C++ tooling for Eclipse. Projects are easier to organize and commit correctly in Eclipse than when set up by hand.

1. The Basic Layout

The example below uses a project named `lastlabw01`, with a `src` folder containing two subfolders, `driver` and `vehicles`. This is the same example used in the Java version of this document, so the layout should look familiar. The project, folder, and file names will vary depending on the assignment.

```
lastlabw01/  
  src/  
    driver/  
      Driver.cpp  
      Driver.h  
    vehicles/  
      Vehicle.cpp  
      Vehicle.h
```

Eclipse CDT's Project Explorer shows this same tree. Unlike Java's Package Explorer, it does not collapse nested folders into a single line, so what you see in Eclipse and what you see in a plain file manager already match. There is no separate view to reconcile.

Table 1. What each folder or file is for.

Location	What It Is
<code>lastlabw01/</code>	The top-level project folder. Name it exactly as the assignment instructs.
<code>lastlabw01/src/</code>	Holds only subfolders. Do not place .cpp or .h files directly inside src.

Location	What It Is
<code>lastlabw01/src/driver/</code>	A subfolder inside <code>src</code> . Folder names are lowercase by convention, though C and C++ do not enforce this.
<code>lastlabw01/src/driver/Driver.cpp</code>	A source file, placed inside its folder along with its header.
<code>lastlabw01/src/driver/Driver.h</code>	The header file for <code>Driver.cpp</code> , declaring what it provides to other files.
<code>lastlabw01/src/vehicles/</code>	Additional folders are additional subfolders inside <code>src</code> .
<code>and so on ...</code>	Repeat this pattern for every folder and source file in the project.

2. Folder and File Rules

Folder names: Unlike a Java package, a C or C++ folder name has no special meaning to the compiler. Lowercase names are still a common convention, and keeping them consistent with your file names makes the project easier to read.

Header guards: Every header file should protect itself from being included more than once. Use an include guard (`#ifndef`, `#define`, `#endif`) or `#pragma once` at the top of each `.h` file.

build folder: Not required in your submission. This is where compiled object files and the final program end up, whether you compile by hand, with a Makefile, or with CMake. It is fine to include it, but it is unnecessary, and many developers add it to `.gitignore` instead.

.project and .cproject files: Not required either. Eclipse CDT creates these to store its own project settings. It is fine if you include them.

Version control: Committing an Eclipse CDT project with Git usually captures everything needed. If you use a different IDE, double-check what it is tracking before you commit, and consider ignoring the build folder along with any compiled files such as `.o` or `.exe`.

3. Using a Non-Eclipse IDE or Editor

If you are not using Eclipse, you do not need to reproduce Eclipse's Project Explorer icons or metadata files. Your file manager or IDE's file tree only needs to match the tree above: a project folder containing a `src` folder, with one subfolder per component, and your `.cpp` and `.h` files inside those folders.

A note on larger, real-world projects: Once a project grows past a single assignment, many C and C++ developers split src and include into separate top-level folders, so header files (the public interface) are kept apart from source files (the implementation). They also add a build system file, such as a CMakeLists.txt or Makefile, so the project can be compiled with one command instead of typing the compiler flags by hand. Neither is required for this course's assignments, but if you continue with C or C++ after this class, this is the structure you are most likely to run into.